

Terrain Engine

A Real-Time OpenGL Terrain, Skybox, and Water Scene

Melvyn KUI

Student No. 2025280173

May 30, 2026

1 Overview

Terrain Engine is a real-time 3D scene written in C++ with the OpenGL 3.3 core profile. The viewer spawns above an island and flies freely through a world made of three cooperating subsystems: a height-map terrain with detail multi-texturing, a manually mapped skybox, and an animated, reflective water plane. The application is a single flat render loop (no scene graph) so that the order and the GL state of each pass are fully explicit.

2 Implementation Details

2.1 Terrain

The terrain mesh is generated at start-up from `heightmap.bmp`. Each grayscale pixel (i, j) becomes a vertex

$$x = (i - \frac{W}{2}) s_{xz}, \quad y = \frac{h(i,j)}{255} s_y + y_0, \quad z = (j - \frac{H}{2}) s_{xz},$$

and adjacent samples are stitched into two triangles per grid cell using an index buffer. The color texture is sampled with the natural grid UVs, while a second *detail* texture is tiled (`uDetailTiling`) to add high-frequency surface variation. The two textures are combined in the fragment shader with the *signed-add* rule, the modern GLSL equivalent of the fixed-function `GL_ADD_SIGNED` mode recommended in the brief:

```
vec3 combined = clamp(baseColor + detailColor - 0.5, 0.0, 1.0);
```

Subtracting 0.5 keeps the average brightness unchanged and avoids the colour clamping artefacts that `GL_ADD` or `GL_MODULATE` would introduce.

The terrain is also lit by a directional sun. Per-vertex normals are computed once at mesh-build time from central differences of the height field, $\mathbf{n} = \text{normalize}(-\partial y / \partial x, 1, -\partial y / \partial z)$, and uploaded as a third vertex attribute. The fragment shader applies an ambient term plus a Lambertian diffuse term, $\text{light} = 0.4 + 0.7 \max(\mathbf{N} \cdot \mathbf{L}, 0)$, so slopes facing the sun brighten and those facing away fall into soft shade while staying visible.

2.2 Skybox

Rather than an OpenGL cubemap, the sky is built from five independent 2D textures mapped onto the inner faces of a cube. This deliberately follows the design brief: the sixth (bottom) face is reserved for the water, and the water texture has a different resolution from the sky faces, which

a cubemap cannot mix. Each face is drawn with a `faceType` uniform that selects the correct sampler, plus per-face `texRotX/Y/Z` uniforms that rotate the UVs in 90° steps so every image appears upright and seamless. Visible seams at the cube edges are removed by setting the wrap mode to `GL_CLAMP_TO_EDGE`.

2.3 Water

The bottom plane samples a 128×128 wave texture with `GL_REPEAT` and a tiling factor well above 1.0 so the low-resolution image is not stretched. Animation is achieved by scrolling the UVs every frame by an accumulated `uWaveShift` (advanced by $\text{speed} \times \Delta t$ so the motion is frame-rate independent):

```
vec2 uv = vec2(TexCoords.x * uWaterTiling + uWaveShift,
               TexCoords.y * uWaterTiling + 0.5 * uWaveShift);
```

The surface is alpha-blended over the scene and tinted slightly toward a sea-blue. A distance-based `smoothstep` fade makes the water more transparent near the camera (revealing the reflection) and opaque toward the horizon, which hides the far edge of the plane.

2.4 Reflections

Two reflections are rendered. The *sky* reflection re-draws the skybox cube scaled by $(1, -1, 1)$ below the water line with depth writes disabled. The *terrain* reflection multiplies the terrain model matrix by a mirror transform about the water plane, $T(0, 2y_w, 0) \cdot S(1, -1, 1)$. Because the negative scale reverses triangle winding, the front-face convention is temporarily switched to `GL_CW` for that draw and restored afterwards. Each terrain pass is bounded by a user clip plane fed through `gl_ClipDistance[0]`: the reflection keeps only geometry below the water surface, and the real pass keeps only geometry above it.

2.5 Camera and asset paths

Movement uses a standard Euler fly-camera (`camera.h`) driven by WASD plus Space/Shift, mouse-look, and scroll-to-zoom. As required by the brief, motion is *smooth*: instead of teleporting the position each frame, the camera keeps a velocity vector that exponentially approaches the target speed while a key is held (gradual acceleration to a cruising speed) and decays back to zero when released (gradual deceleration). The exponential approach $v += (v_{\text{target}} - v)(1 - e^{-r \Delta t})$ makes the feel frame-rate independent. All asset and shader paths are resolved from a compile-time `PROJECT_ROOT` string injected by CMake, so the executable loads its resources correctly regardless of the current working directory.

3 Results

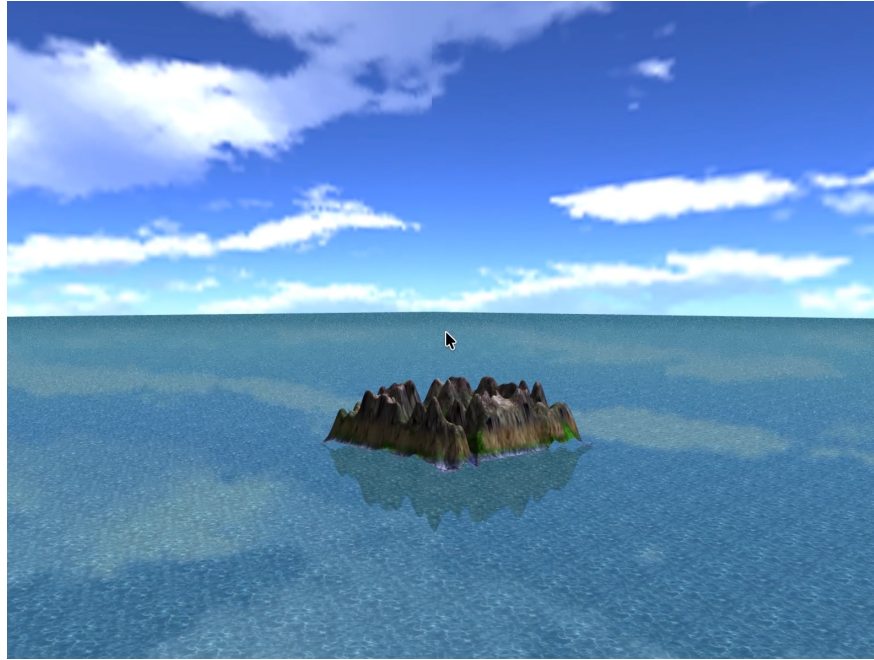
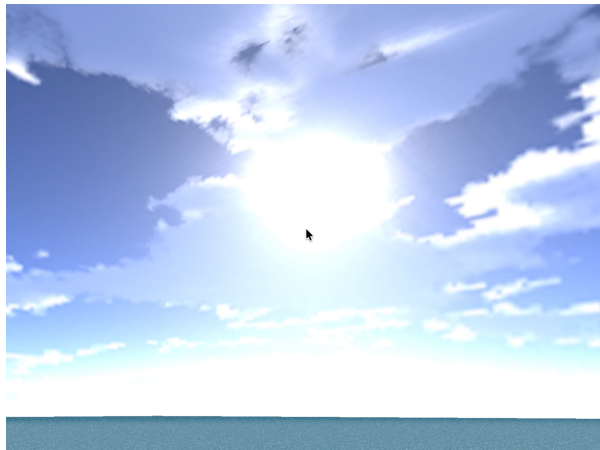
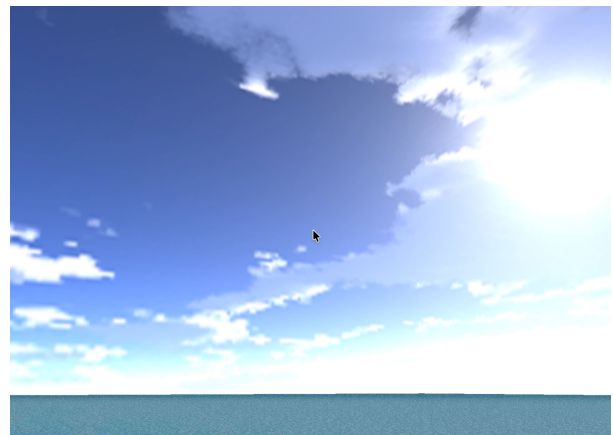


Figure 1: Overview of the scene: the island terrain sits on the animated water under the skybox, with the island visible as a reflection in the water.



(a) The round sun and clouds in the sky.

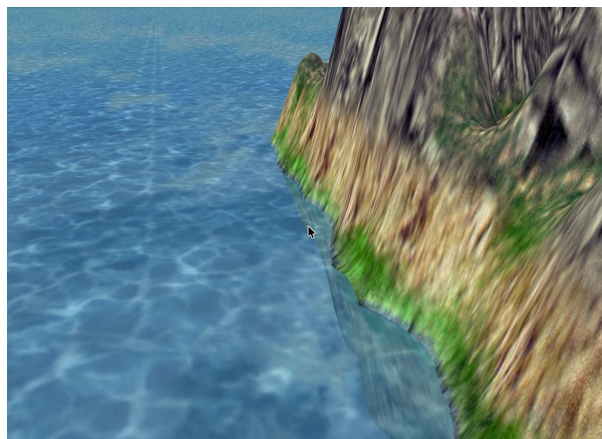


(b) Sky with no visible cube edges/seams (GL_CLAMP_TO_EDGE).

Figure 2: Skybox: the sun and clouds are part of the sky images, and the box edges are hidden so the sky reads as continuous.

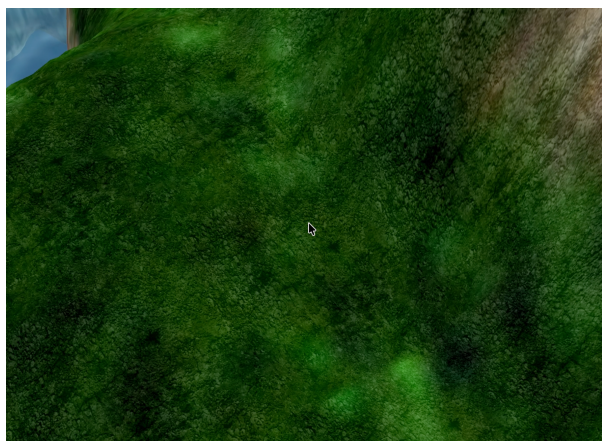


(a) Terrain mirrored in the water surface.

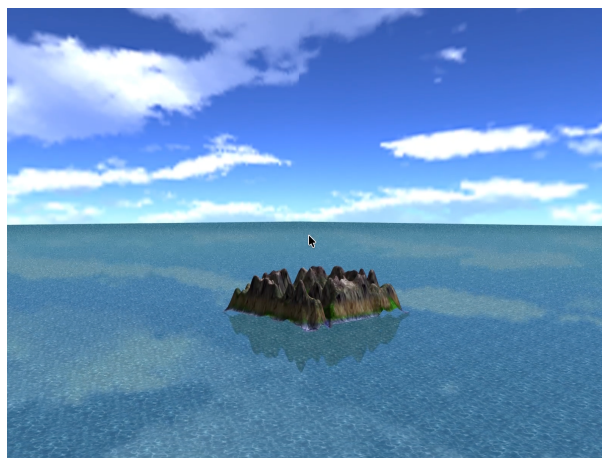


(b) Close-up of the tiled, scrolling water surface at the shore.

Figure 3: Water: a reflective, animated surface. The reflection is the scene redrawn mirrored under the water line and blended through the surface.



(a) Color + detail multi-texturing close up.



(b) Terrain relief lit by the directional sun.

Figure 4: Terrain: height-map geometry with the color and detail textures combined, and slopes shaded by the sun direction.

4 Challenges & Solutions

Putting the right image on the right cube face

This was the hardest part for me. I have five sky images and a cube, and at first almost every face was either showing the wrong image or showing the right image sideways or upside down. I ended up giving each face a `faceType` id plus three rotation uniforms (`texRotX/Y/Z`) and rotating the texture coordinates in 90° steps in the fragment shader until each one looked right. For example the left face needed a 90° Y plus 180° X rotation and the top face needed 180° on both Y and Z. It was mostly trial and error, comparing the seams between neighbouring faces until the clouds lined up.

Hiding the edges of the box

At first the borders of the cube were clearly visible as hard lines, which gave away that the sky is just a box. This turned out to be the texture wrapping mode: with the default clamp the edge texels bled, but switching every face to `GL_CLAMP_TO_EDGE` made the seams disappear.

Flipping the box for the reflection

For the water I redraw the scene mirrored under the surface. The skybox reflection is the same cube scaled by $(1, -1, 1)$, and the terrain reflection is the terrain matrix times a mirror about the water plane. The annoying bug was that the mirrored terrain disappeared, because the negative scale reverses the triangle winding so it was being back-face culled. Switching `glFrontFace` to `GL_CW` just around the mirrored draw (and back to `GL_CCW` after) fixed it. I also had to disable depth writes for the reflection and turn on blending so the water and the reflection mix instead of one hiding the other.

Cutting the terrain at the water line

The reflected terrain should only appear below the water and the real terrain only above it, otherwise they poke through each other at the shore. I used `gl_ClipDistance[0]` with a `uClipPlane` uniform, flipping the plane direction between the two passes.

Making speed frame-rate independent

My first version moved the camera and scrolled the waves by a fixed amount per frame, so everything ran faster on a faster machine. Multiplying by `deltaTime` (and, for the camera, ramping the velocity with an exponential factor) made the motion consistent.

Known limitation. The sky images are only 256×256 and are stretched over a large cube, so the sky still looks a bit soft up close. Higher-resolution face images would fix this without any code change.

5 Environment Setup

Operating system	macOS (Darwin 25.4, Apple Silicon)
Compiler / standard	Apple Clang, C++14
Build system	CMake (≥ 3.5)
Windowing / input	GLFW 3.4 (static <code>libglfw3.a</code>)
GL loader	GLAD (OpenGL 3.3 core)
Math library	GLM
Image loading	stb_image

GLAD, GLFW, GLM, and stb_image are vendored under `external/`; only CMake must be installed to build. The application is built with `cmake -B build && cmake --build build` and runs as `./terrainEngine`.

6 Bonus & References

Features beyond the basic requirements

- **Dual reflection** of both the sky and the terrain in the water, each clipped to the correct half-space with `gl_ClipDistance`.
- **Distance-based water transparency** using a **smoothstep** fade, so the water reveals the reflection up close and hides the plane edge at range.
- **Frame-rate-independent** wave animation via Δt accumulation.
- **Portable asset resolution** through a compile-time `PROJECT_ROOT` define, allowing the binary to run from any directory.
- **Self-contained build**: all third-party dependencies vendored in-repo.
- **4× MSAA** for anti-aliased geometry edges.
- **Directional sun lighting** on the terrain from height-field normals.

Possible future improvements

- **GPU-displaced** water with real normals for specular sun glint, instead of a scrolling texture.
- **Fresnel-weighted** reflection blending for a more physically convincing water surface.
- **Frustum culling / LOD** on the terrain grid for larger maps.
- Higher-resolution or procedurally generated sky.
- **Shadow mapping** so the terrain casts shadows under the sun, instead of diffuse-only shading.
- **Specular highlights** and a simple Blinn–Phong term on the terrain, and a sun direction tied to the bright spot baked into the skybox so shading and sky agree.
- **Normal mapping** on the terrain detail texture for fine surface relief that does not need extra geometry.
- **Height-based texturing** (sand near the water, grass mid-slope, rock/snow on peaks) blended by altitude and slope, instead of one colour map.
- **Screen-space reflections** or planar-reflection FBO for the water, removing the duplicated geometry passes used today.
- **Distance fog / atmospheric haze** to blend the terrain edge into the sky and hide the far clip boundary.
- **HDR + bloom** tone mapping so the sun and its water glint can bloom.

- **Underwater fog / refraction** when the camera dips below the water line.
- **Day–night cycle**: animate the sun direction and tint ambient/sky colour over time.
- **Tessellation or geo-mipmapping** to add terrain detail near the camera while keeping distant tiles cheap.
- **Restore an optional HUD/minimap** (re-add text or an ImGui debug overlay) for position read-out and live tuning of the scene constants.

References

1. Joey de Vries, *LearnOpenGL* — <https://learnopengl.com/> (camera, shaders, text-rendering, skybox tutorials).
2. GLFW 3.4 documentation — <https://www.glfw.org/docs/latest/>.
3. OpenGL Mathematics (GLM) — <https://github.com/g-truc/glm>.
4. Sean Barrett, `stb_image` — <https://github.com/nothings/stb>.
5. Course design brief, `doc/TerrainDoc.md` (skybox / water / terrain spec).